

Spans would do. ImageSpan solves this problem by allowing us to put any Drawable or Bitmap in between the text.

```
SpannableString image = new SpannableString("String with <-
image");
int start = image.length() - "<- image".length();
image.setSpan(new ImageSpan(this, R.mipmap.ic_launcher),
start, start+1, 0);
textView.setText(image);
```

Although it is quite easy to set the Span, we need to be careful here, because the image could just be drawn over all the text we have in that Span. We need to be sure that we are not applying the Span over any text, unless it is intentional.

TextAppearanceSpan

The example we have shown you in Figure 1 is a bit complex because it uses more than one span to achieve that effect. But we will talk about TextAppearanceSpan only. You can see the other code and follow it on your own as an exercise to know what it does.

TextAppearanceSpan can be created in two ways. One is to construct the object from a style in your project, and the other is to manually provide values in the constructor. The code is as follows:

```
SpannableString appearance = new SpannableString("String with
different appearance and link appearance");
appearance.setSpan(new TextAppearanceSpan("serif", 0, 30,
    colourStateList.valueOf(colour.WHITE),
colourStateList.valueOf(colour.BLUE)), 22,
32, 0);
appearance.setSpan(new BackgroundColourSpan(colour.RED), 22,
32, 0);
appearance.setSpan(new TextAppearanceSpan("serif", 0, 30,
    colourStateList.valueOf(colour.WHITE),
colourStateList.valueOf(colour.BLUE)),
37, appearance.length(), 0);
appearance.setSpan(new ClickableSpan() {
    @Override
    public void onClick(View widget) {
        Toast.makeText(widget.getContext(), "That is a link",
        Toast.LENGTH_SHORT).show();
    }
}, appearance.length() - "link appearance".length(),
appearance.length(), 0);
textView.setMovementMethod(new LinkMovementMethod());
textView.setText(appearance);
```

We have manually provided values to the TextAppearanceSpan here. The first parameter in the constructor is the font-family—it must be one of sans-serif, serif or monospace. Next is the style of font—we have passed 0 here, but you can pass any style constant from the *Typeface*

class such as *Typeface.BOLD* or *Typeface.BOLD_ITALIC*. Next is the size of the font. Keep in mind that this is the absolute size. The next parameter is the colorStateList for the colour of the text. For those who do not know about colorStateList, it is a StateList which changes colour according to the state of the widget such as whether it is pressed or focused. We want the same colour for all the states, so we use the convenient static method in the class to construct a colorStateList with only one colour. The fourth and last parameter is the same as the third parameter—it is a colorStateList. But it is applied only on the links inside the text.

URLSpan

URLSpan is an implementation of ClickableSpan, which opens the provided link upon clicking in the Span.

```
SpannableString url = new SpannableString("Click here to open
a URL");
url.setSpan(new URLSpan("http://www.samvidinfotech.in"), 0,
"click here".length(), 0);
textView.setMovementMethod(new LinkMovementMethod());
textView.setText(url);
```

The only parameter URLSpan requires is the link to open when clicked. And you should not forget to set the movement method for LinkMovementMethod; otherwise, it won't work.

Creating your own Spans

You may have noticed that the first parameter in the setSpan method takes in an object. Anything can be provided as a Span there and it will work. Spans in the framework are generally divided into two categories—the ones extending CharacterStyle and those implementing ParagraphStyle. CharacterStyle Spans affect the text of the Span at the character level, such as changing the text colour, whereas ParagraphStyle Spans affect the text more at the block level. These affect the paragraph, such as providing a leading margin on the first line of a paragraph.

You aren't supposed to extend or implement these basic Spans, but you can extend or implement one of the Spans that provide you more control over how the text is drawn, such as ReplacementSpan when you need absolute control over the drawing of text. We are not going to get into details but here is a simple class which draws a border around the text on which it is applied. Use this as a reference and dig a bit deeper into the topic to get the exact custom style you or your designer wants.

```
public class TextBorderSpan implements LineBackgroundSpan {
    private Path mPath;

    @Override
    public void drawBackground(Canvas canvas, Paint paint,
    int left, int right, int top, int baseline, int bottom,
```